

Module 2 – Express Framework

1. Intro to Express

What is Express?

Express is a lightweight and flexible Node.js web application framework used to build APIs and backend applications.

Why Express?

- Simplifies server creation
 - Handles routing easily
 - Middleware support
 - Fast development
-

2. Routing

What is Routing?

Routing defines how an application responds to client requests for different URLs.

Types of Routes

- GET – Read data
- POST – Create data
- PUT – Update data
- DELETE – Remove data

Example:

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Home Page");
});
```

3. Middleware

What is Middleware?

Middleware functions are functions that execute during the request-response cycle.

Why Middleware?

- Modify requests and responses
- Add authentication
- Handle logging
- Error handling

Example:

```
app.use((req, res, next) => {  
  console.log("Request received");  
  next();  
});
```

4. Request & Response

Request Object

Contains data sent by the client.

Response Object

Used to send data back to the client.

Example:

```
app.get("/user", (req, res) => {  
  res.json({ name: "Ali", age: 20 });  
});
```

5. Error Handling

What is Error Handling?

Error handling manages unexpected issues in the application.

Why It Matters

- Prevents app crashes
- Improves stability
- Better user experience

Example:

```
app.use((err, req, res, next) => {  
  res.status(500).send("Server Error");  
});
```

6. Key Concepts to Remember

- Express simplifies backend development
 - Routing handles URLs and requests
 - Middleware controls request flow
 - Request carries client data
 - Response sends data back
 - Error handling ensures stability
-

7. Summary

Express is a powerful framework that makes backend development faster and easier. It provides routing, middleware, and error handling features that help build scalable and maintainable server-side applications.